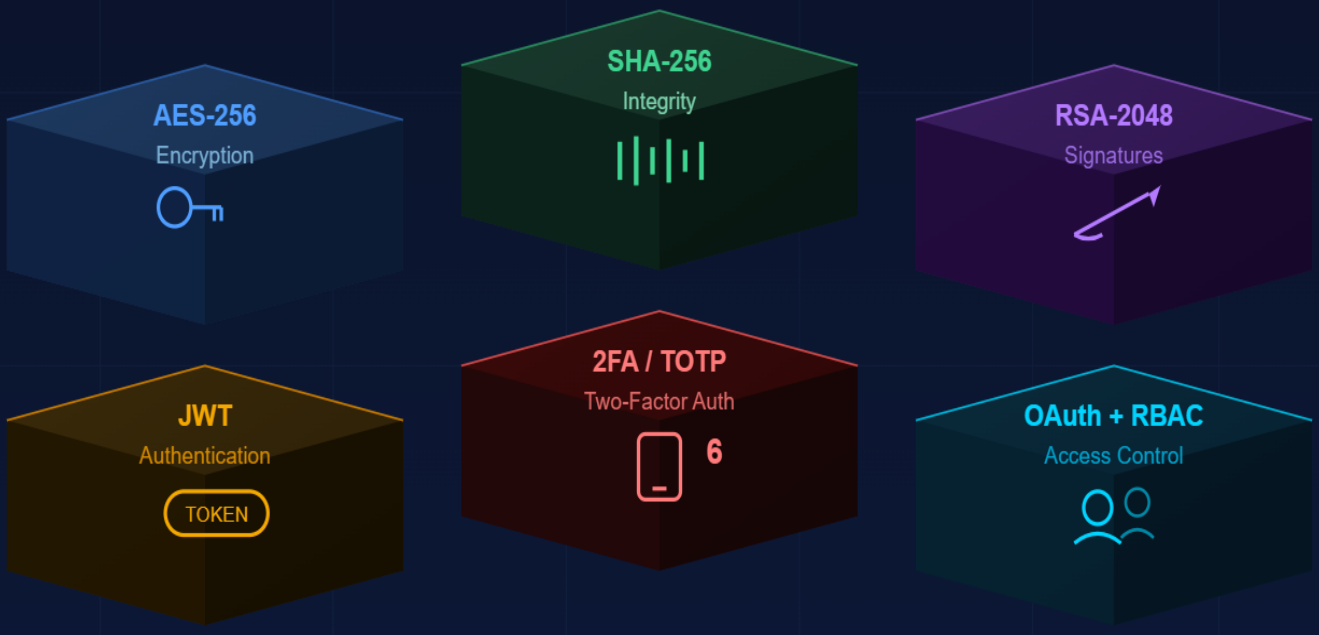
Secure Document Vault

SYSTEM



Alexandria National University

Faculty of Computers and Data Science · Cyber Security

Node.js · React.js · PostgreSQL · bcrypt · HTTPS/TLS

DATA INTEGRITY AND AUTHENTICATION · SPRING 2026

1. Project Overview

The Secure Document Vault System is a full-stack web application designed to demonstrate real-world security practices in document management. The system allows users to securely upload, store, encrypt, digitally sign, and verify documents while enforcing modern authentication and authorization mechanisms.

This project was developed as the final deliverable for the Data Integrity and Authentication course, simulating an enterprise-grade secure platform.

2. Project Objectives

The main objectives of this project are:

- Implement modern authentication mechanisms including JWT, OAuth, and 2FA
- Apply secure password storage using bcrypt hashing with policy enforcement
- Implement Role-Based Access Control (RBAC) with Admin, Manager, and User roles
- Encrypt uploaded documents using AES-256 symmetric encryption
- Generate SHA-256 hashes and RSA digital signatures for integrity verification
- Secure communication using HTTPS with SSL/TLS certificates
- Demonstrate the importance of HTTPS through Wireshark traffic analysis

3. Technology Stack

Layer	Technology	Purpose
Backend	`Node.js + Express.js`	RESTful API server
Frontend	`React.js + Vite`	User Interface
Database	`PostgreSQL + Sequelize`	Data persistence & ORM
Authentication	`JWT + Passport.js`	Token-based auth & OAuth
Encryption	`Node.js crypto (AES-256)`	Document encryption
Hashing	`SHA-256`	Integrity verification
Digital Signatures	`RSA-2048`	Document signing
2FA	`Speakeasy + Google Auth`	Two-factor authentication
HTTPS	`Self-signed SSL (mkcert)`	Secure communication
Password Hashing	`bcryptjs`	Secure password storage

4. System Architecture

The system follows a three-tier client-server architecture:

- Presentation Layer: React.js frontend running on port 5173
- Application Layer: Node.js/Express backend API running on port 5000 (HTTPS)
- Data Layer: PostgreSQL database storing user and document metadata

All communication between layers is secured via HTTPS/TLS. Documents are encrypted at rest using AES-256 before storage on the filesystem.

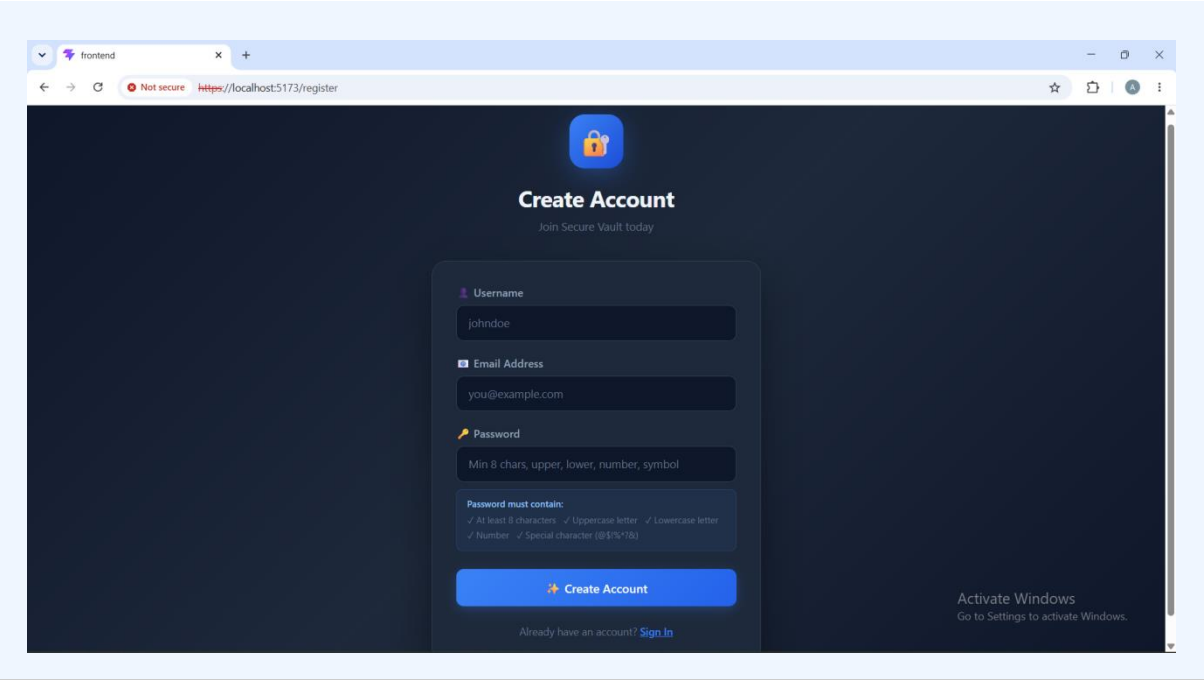
5. Security Features Implementation

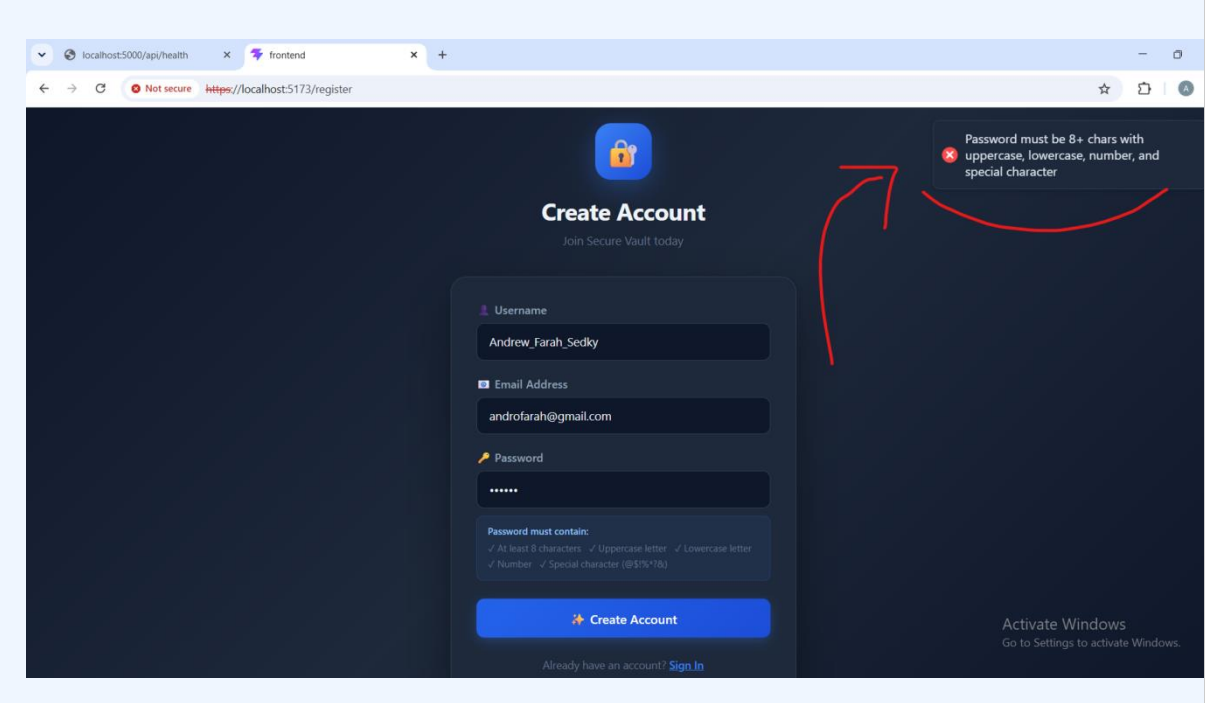
5.1 User Registration & Password Policy

Users register with a username, email, and password. Passwords are validated against a strict policy before hashing:

- Minimum 8 characters
- At least one uppercase letter (A-Z)
- At least one lowercase letter (a-z)
- At least one digit (0-9)
- At least one special character (@$!%*?&)

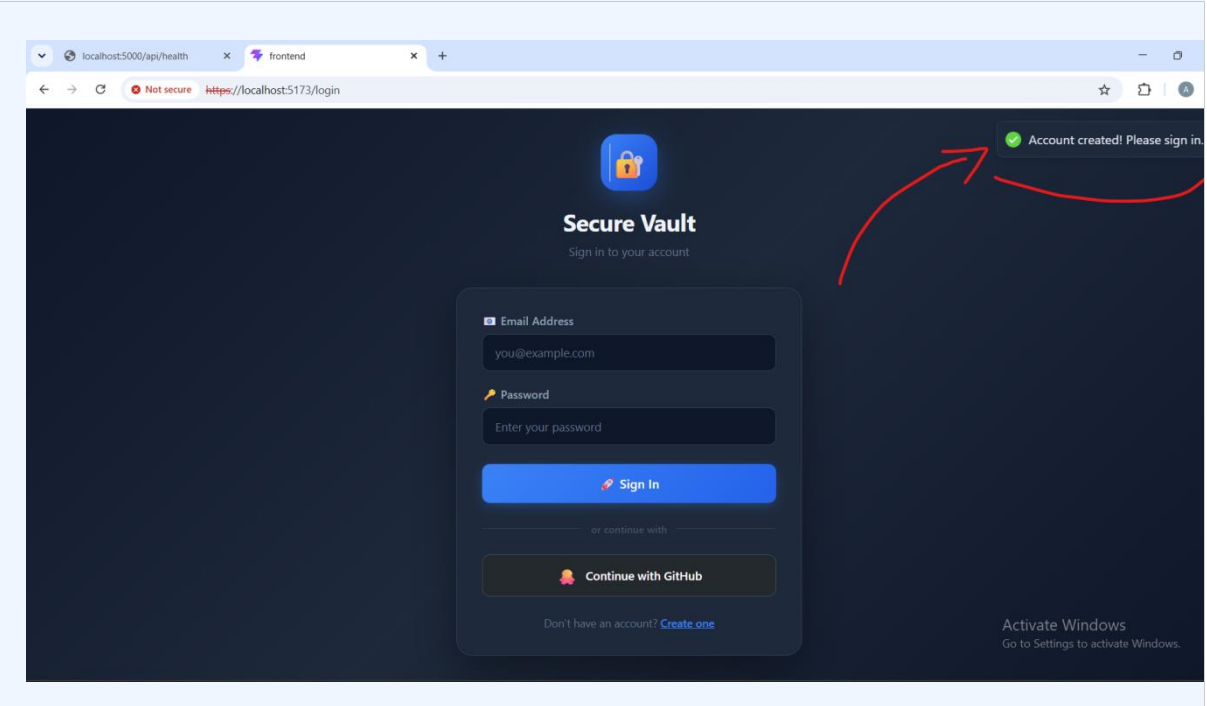
Passwords are hashed using bcrypt with a cost factor of 12 before being stored in the database. Plain-text passwords are never stored.

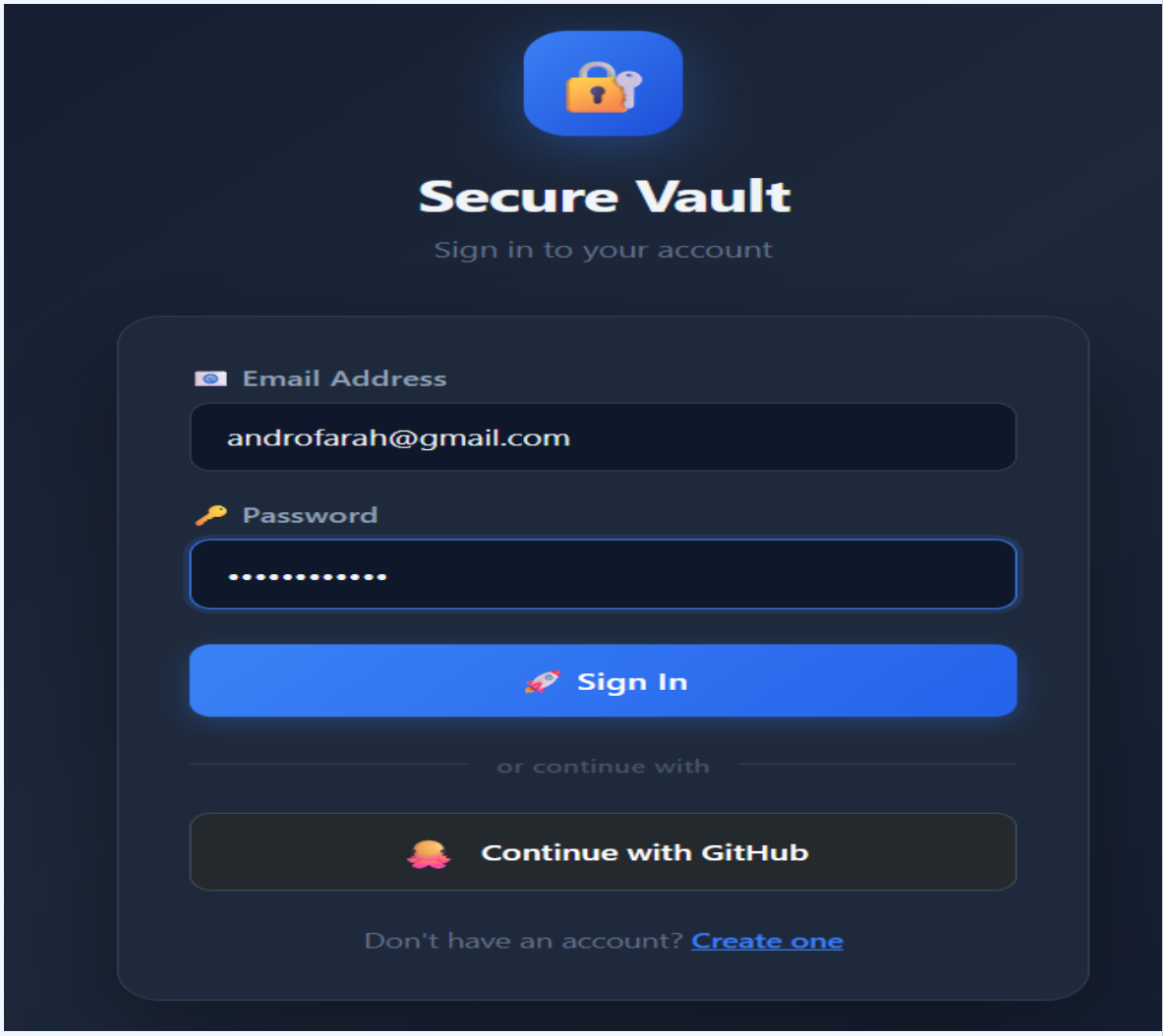


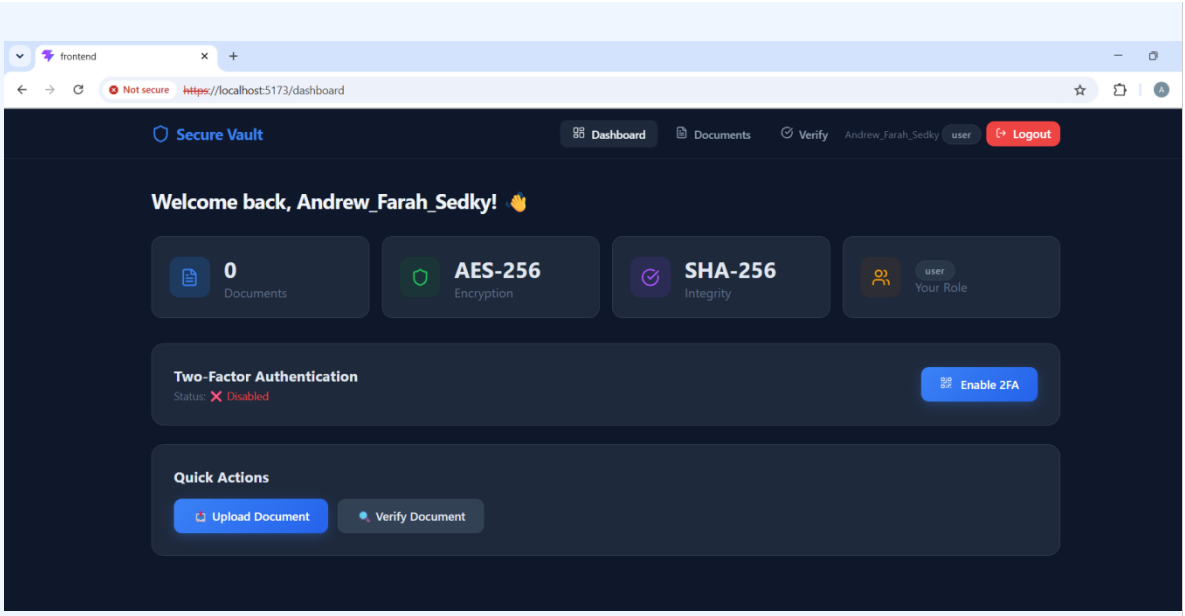


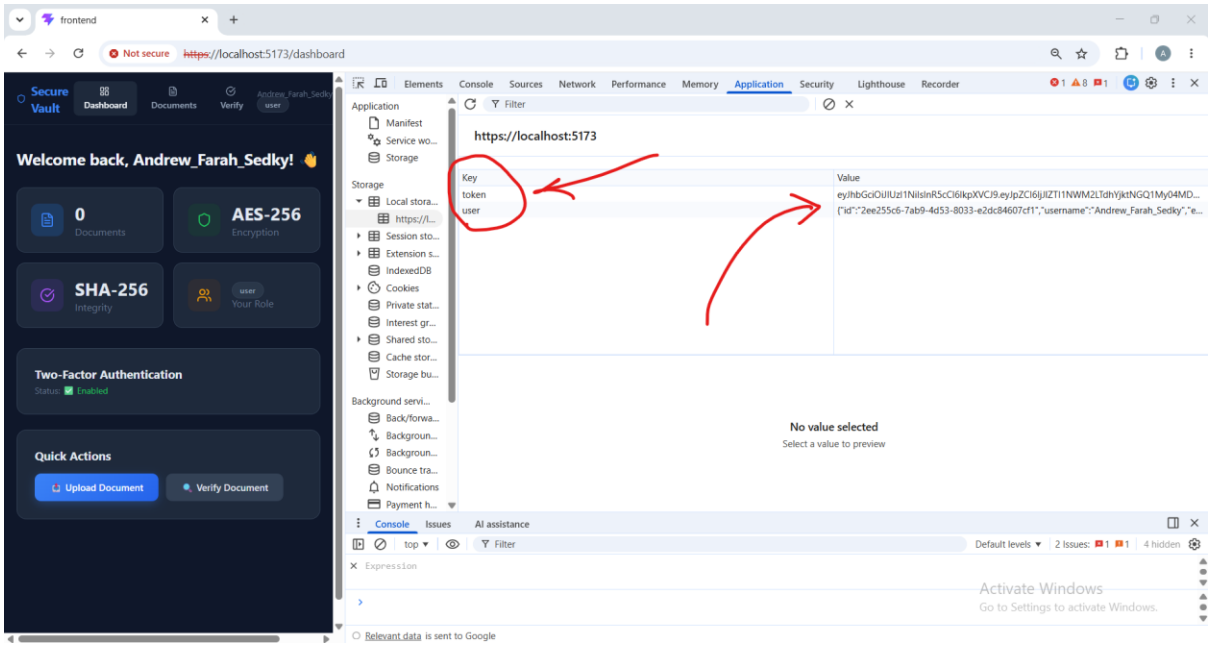
5.2 JWT Authentication

After successful login, the server issues a JSON Web Token (JWT) signed with a secret key. The token contains the user ID and role, and expires after 24 hours. All protected API endpoints require this token in the Authorization header.



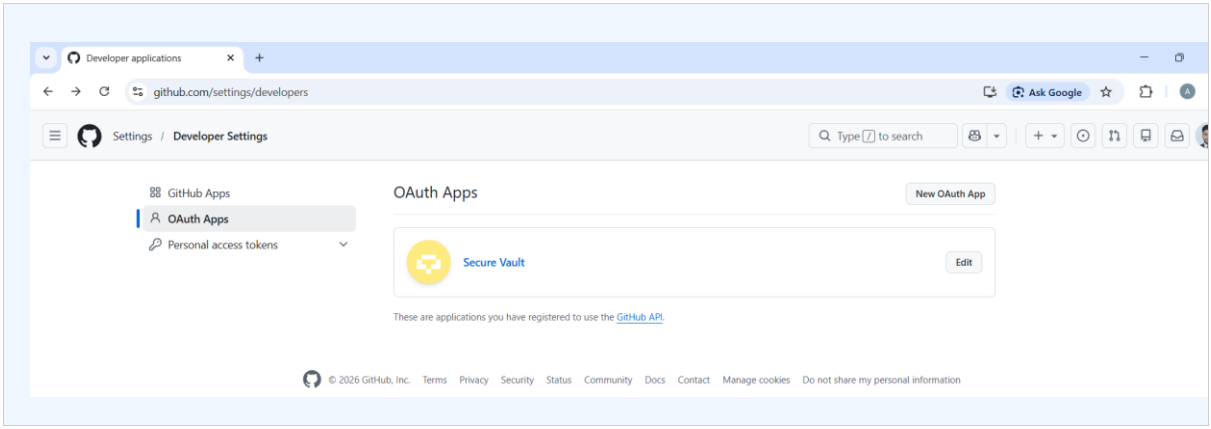

Secure Vault
Sign in to your account
Email Address
androfarah@gmail.com
Password
Sign In
or continue with
Continue with GitHub
Don't have an account? Create one

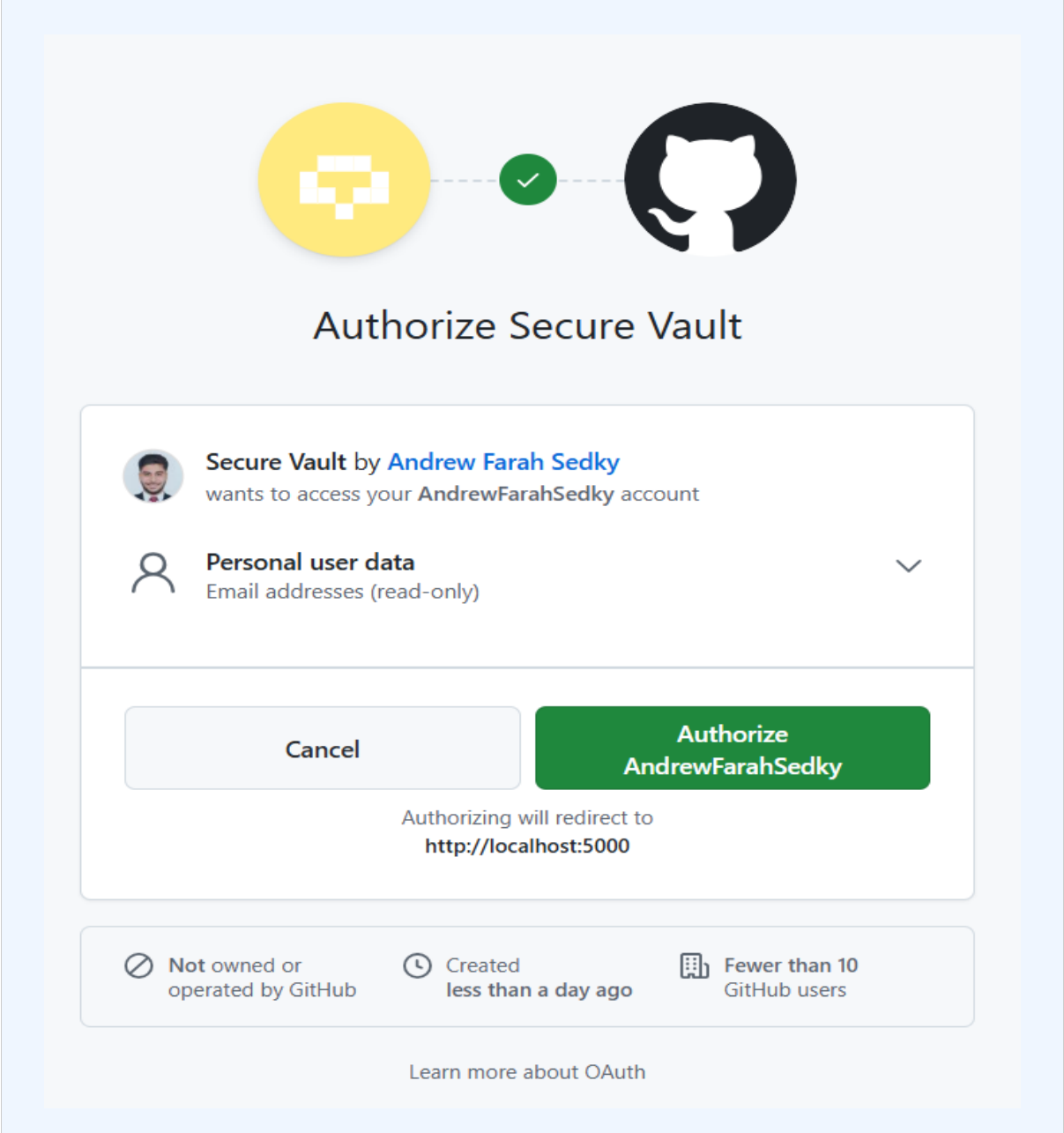

frontend
Not secure
https://localhost:5173/dashboard
Secure Vault
Dashboard
Documents
Verify
Andrew_Farah_Sedky
user
Logout
Welcome back, Andrew_Farah_Sedky!
0
Documents
AES-256
Encryption
SHA-256
Integrity
user
Your Role
Two-Factor Authentication
Status: Disabled
Enable 2FA
Quick Actions
Upload Document
Verify Document

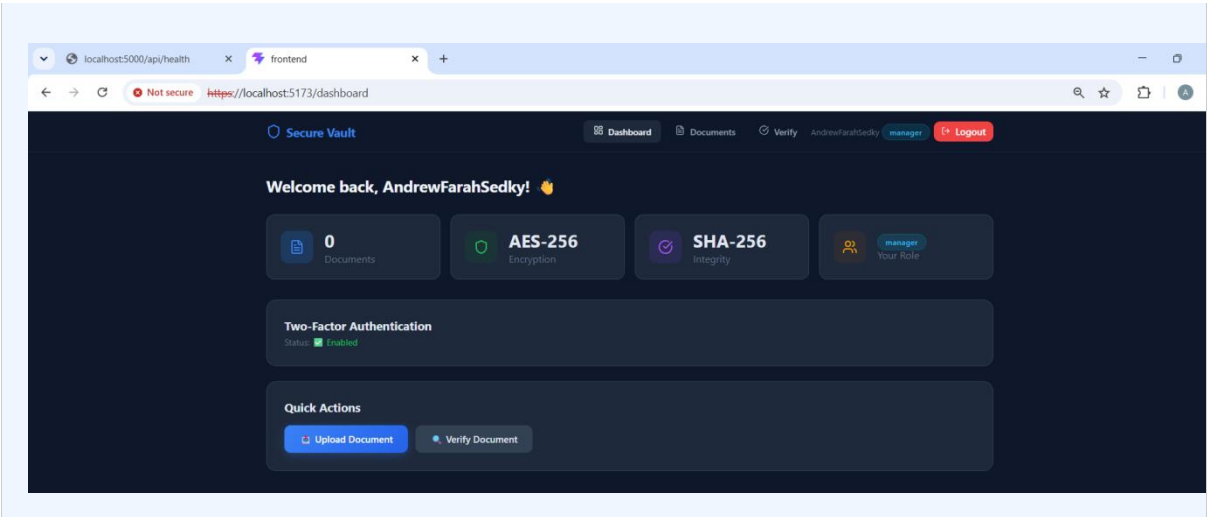


5.3 OAuth Login (GitHub)

Users can authenticate using their GitHub account via OAuth 2.0. The system uses Passport.js with the GitHub strategy. Upon successful OAuth authentication, the server creates or retrieves the user record and issues a JWT token.

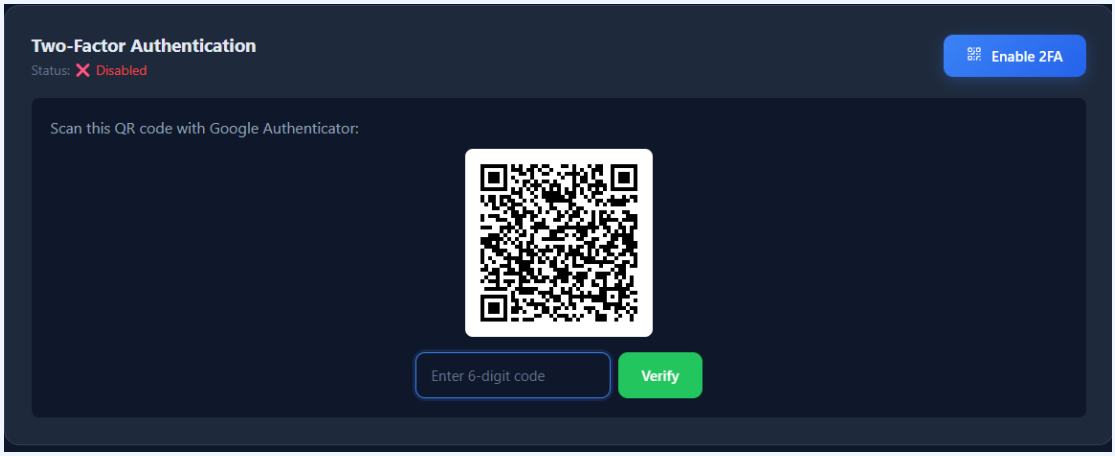


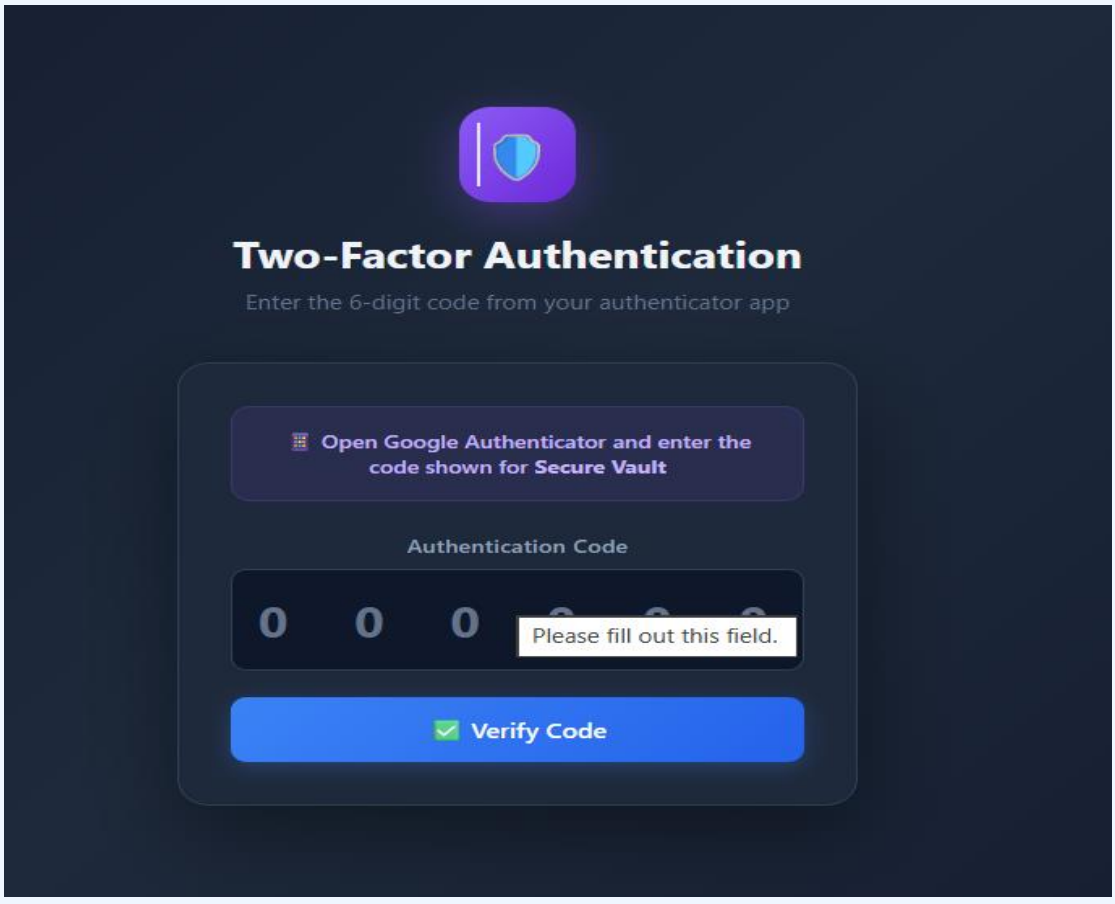

Authorize Secure Vault
Secure Vault by Andrew Farah Sedky
wants to access your AndrewFarahSedky account
Personal user data
Email addresses (read-only)
Cancel
Authorize AndrewFarahSedky
Authorizing will redirect to
http://localhost:5000
Not owned or operated by GitHub
Created less than a day ago
Fewer than 10 GitHub users
Learn more about OAuth

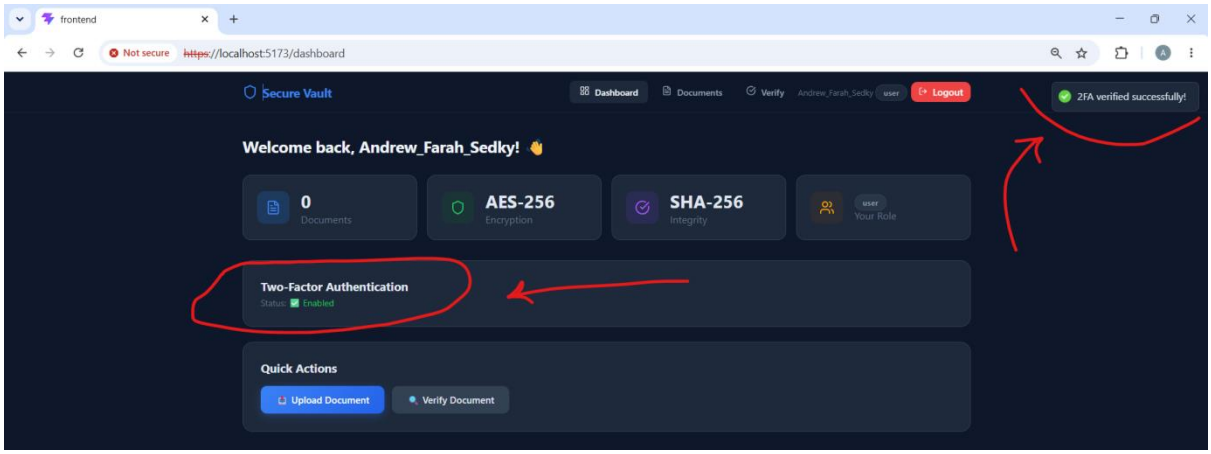

localhost:5000/api/health
frontend
Not secure
https://localhost:5173/dashboard
Secure Vault
Dashboard
Documents
Verify
AndrewFarahSedky
manager
Logout
Welcome back, AndrewFarahSedky!
0
Documents
AES-256
Encryption
SHA-256
Integrity
manager
Your Role
Two-Factor Authentication
Status: Enabled
Quick Actions
Upload Document
Verify Document

5.4 Two-Factor Authentication (2FA)

The system supports TOTP-based Two-Factor Authentication using the speakeasy library, compatible with Google Authenticator. Users can enable 2FA from the Dashboard by scanning a QR code. On subsequent logins, users must enter a valid 6-digit TOTP code.



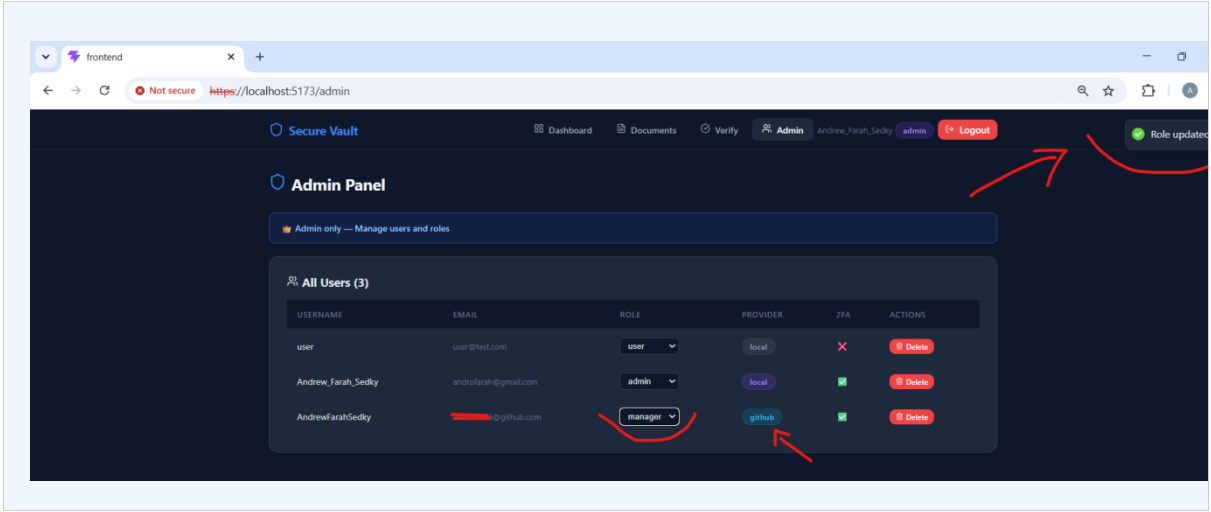


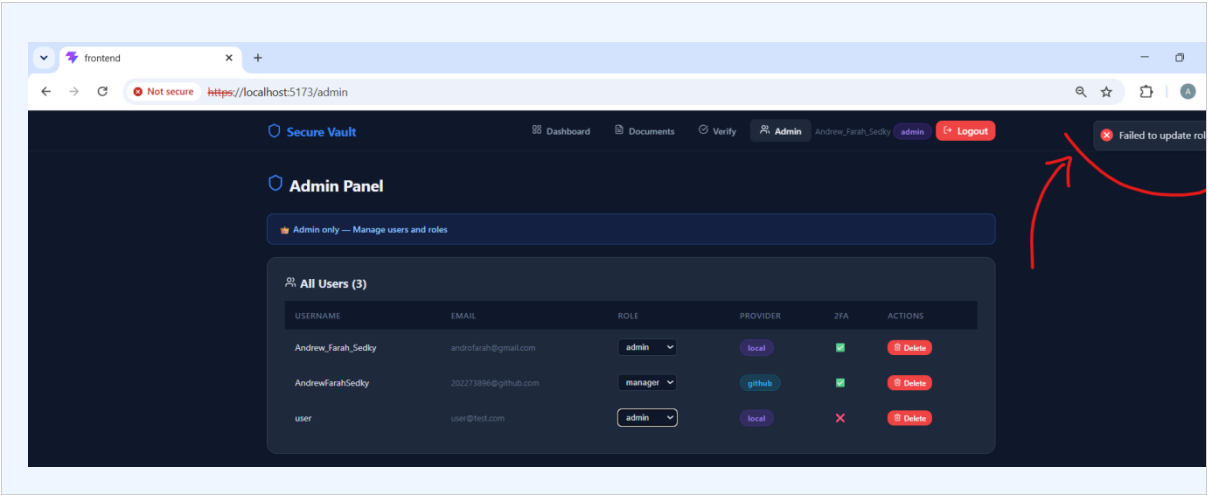


5.5 Role-Based Access Control (RBAC)

The system implements three user roles with different permissions:

Role	Permissions	Access Level
Admin	Manage all users and roles, view all documents	Full system access
Manager	Review and verify documents of all users	Elevated access
User	Upload, view, download, and delete own documents	Standard access

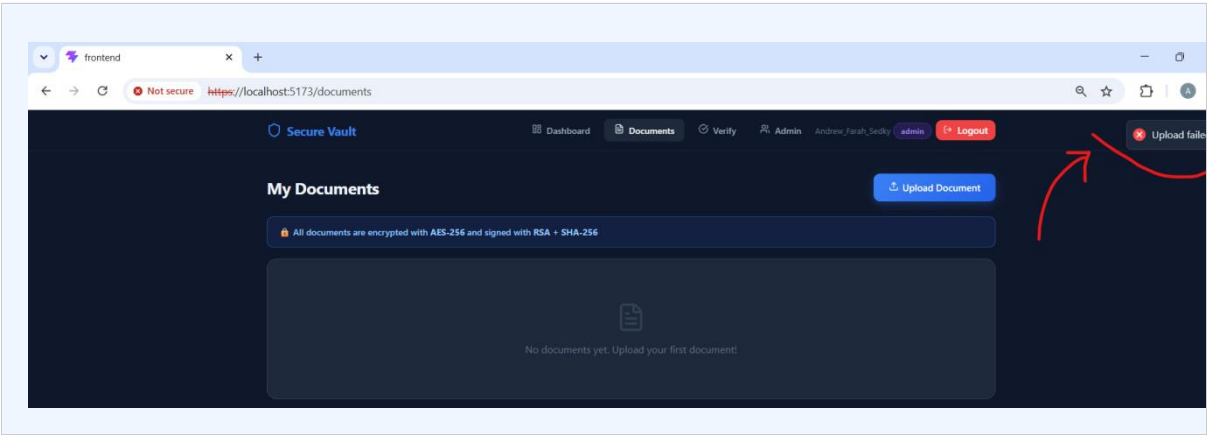


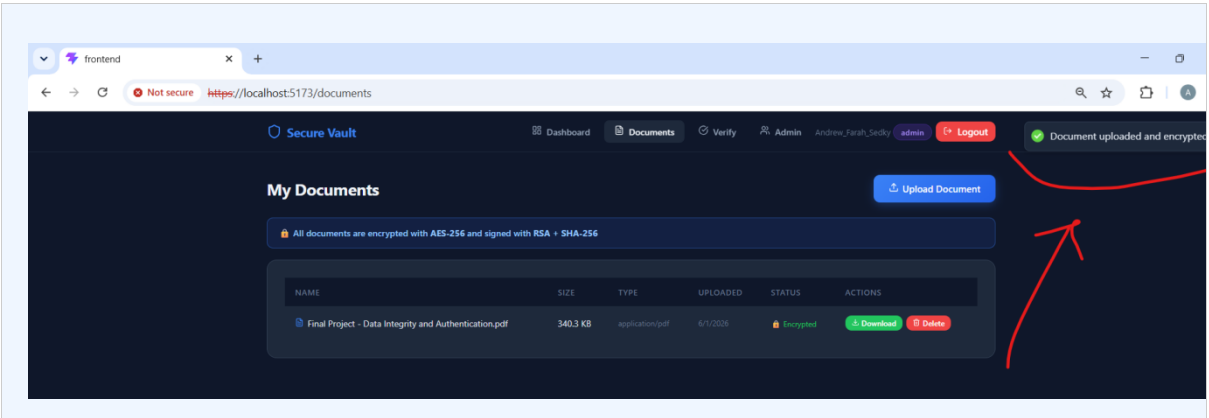


6. Document Management

6.1 Document Upload

Users can upload documents through the web interface. The system validates file types and sizes before processing. Allowed formats include PDF, JPEG, PNG, TXT, DOC, and DOCX. Maximum file size is 10 MB.





6.2 Document Encryption (AES-256)

Every uploaded document is encrypted using AES-256-CBC before being saved to the filesystem. The process works as follows:

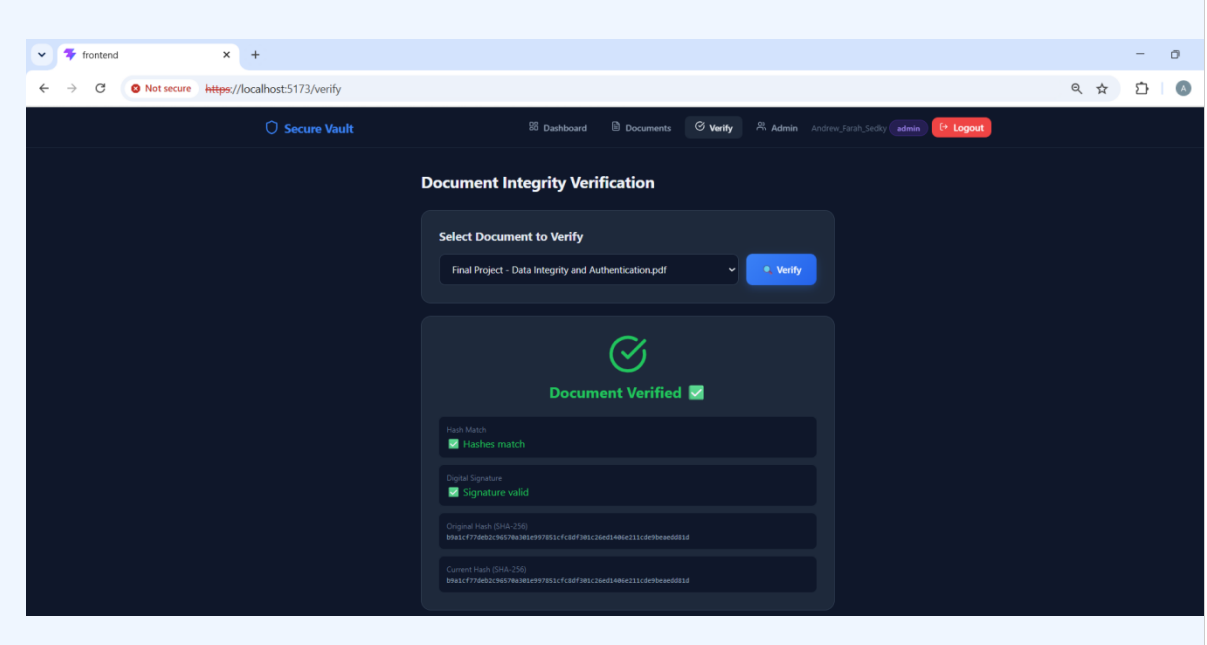
- A random 16-byte Initialization Vector (IV) is generated for each file
- The document is encrypted using AES-256-CBC with the IV and a 32-byte server key
- The IV is prepended to the encrypted file and stored with a .enc extension
- The original unencrypted file is deleted immediately after encryption

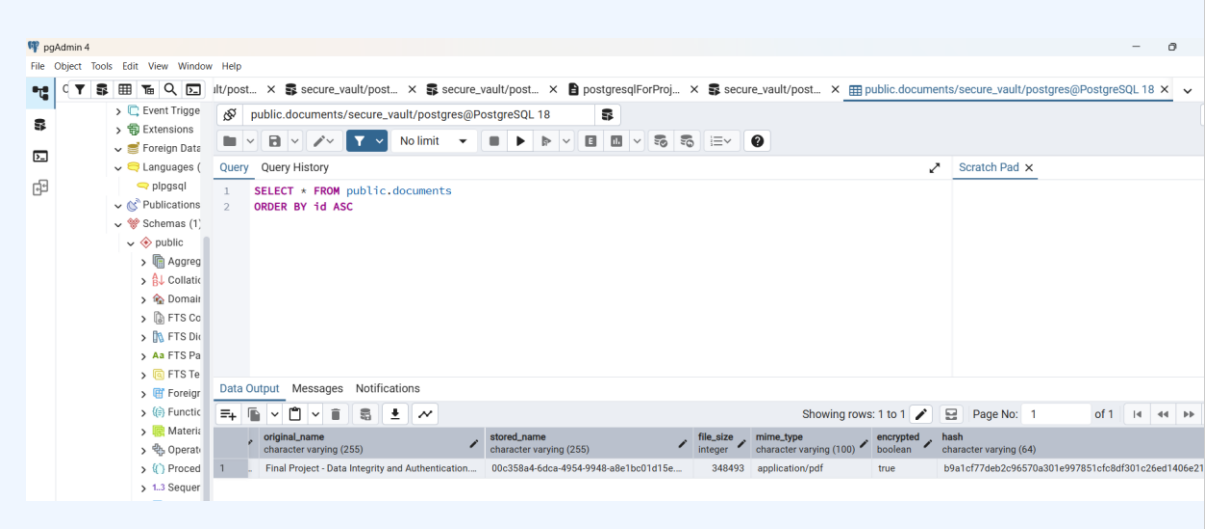
This ensures documents cannot be read directly from storage even if the filesystem is compromised.

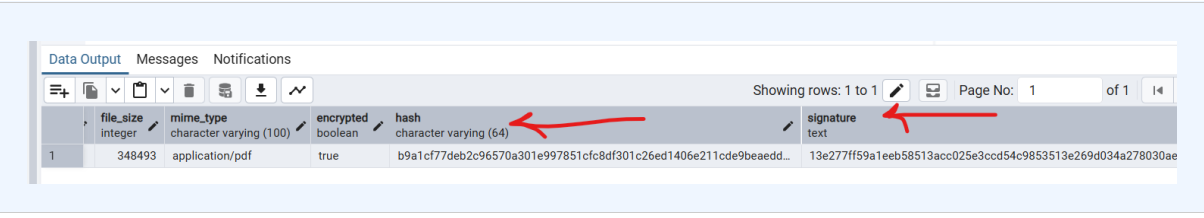
7. Digital Signatures & Integrity Verification

The system implements a complete document integrity verification pipeline using cryptographic techniques:

Step	Description
1. Hash Generation	A SHA-256 hash is computed from the original file before encryption
2. Digital Signing	The hash is signed using RSA-2048 private key (stored in certs/private.pem)
3. Storage	Both hash and signature are stored in the database with the document metadata
4. Verification	On verify request: decrypt file, compute current hash, compare with stored hash, verify RSA signature with public key
5. Tamper Detection	If the encrypted file is modified, the hash will not match — tampering is detected



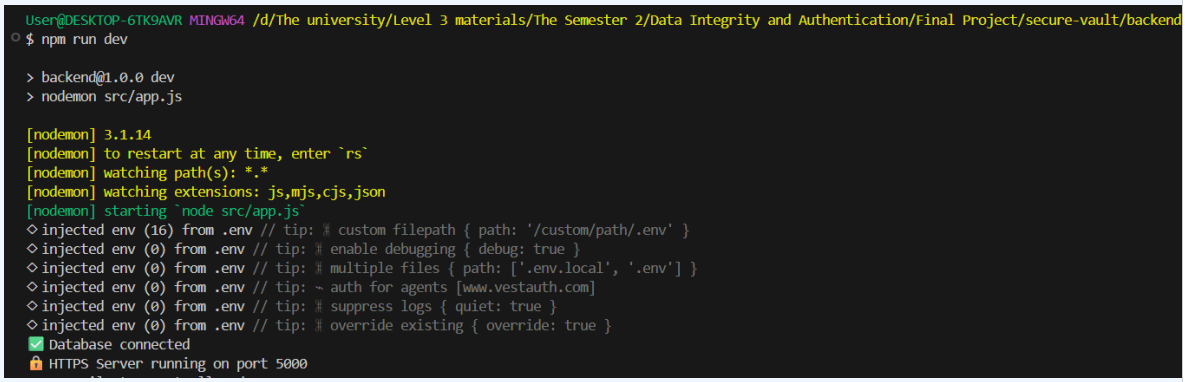


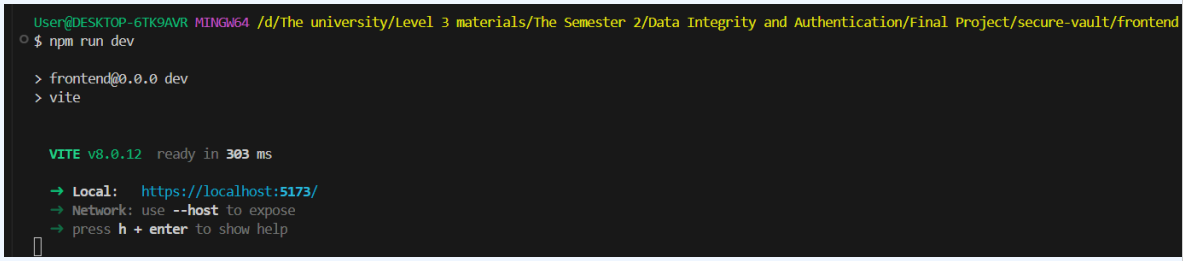


8. HTTPS & Secure Communication

The application is configured to run over HTTPS using a self-signed SSL certificate generated with mkcert. HTTPS ensures that all data transmitted between the client and server is encrypted using TLS, preventing eavesdropping and man-in-the-middle attacks.

- SSL certificate and private key are stored in backend/certs/
- The Node.js HTTPS server loads the certificate at startup
- The browser displays a padlock icon confirming the secure connection



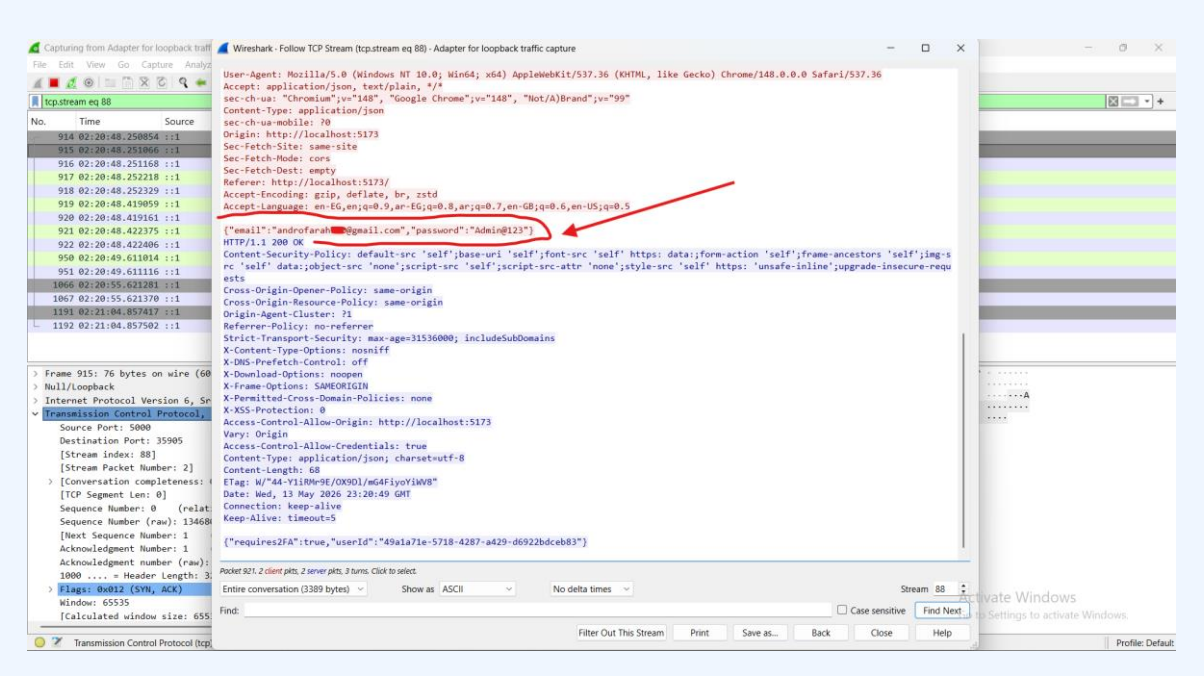


9. MITM Traffic Analysis — Wireshark Demo

To demonstrate the importance of HTTPS, we captured network traffic using Wireshark under both HTTP and HTTPS conditions.

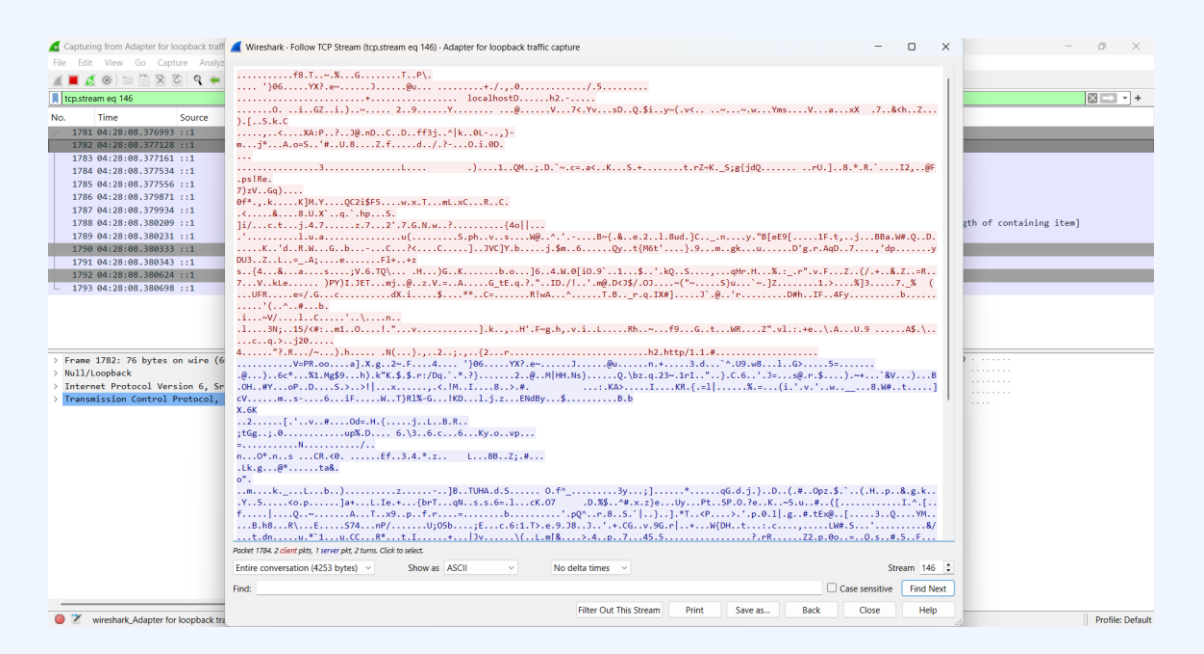
9.1 HTTP Traffic (Before HTTPS)

With the server running on HTTP, login credentials were sent in plain text. Wireshark captured the TCP stream and clearly showed the email and password fields readable in the packet data.



9.2 HTTPS Traffic (After Enabling HTTPS)

After enabling HTTPS, the same login attempt was captured. Wireshark showed only encrypted TLS data — no readable credentials were visible. This confirms that HTTPS effectively protects data in transit against interception.



9.3 Comparison Summary

Aspect	HTTP	HTTPS
Credentials visible	Yes — plain text	No — encrypted
Data encryption	None	TLS/SSL encrypted
MITM vulnerability	High	Protected
Wireshark readable	Fully readable	Encrypted gibberish

10. Database Design

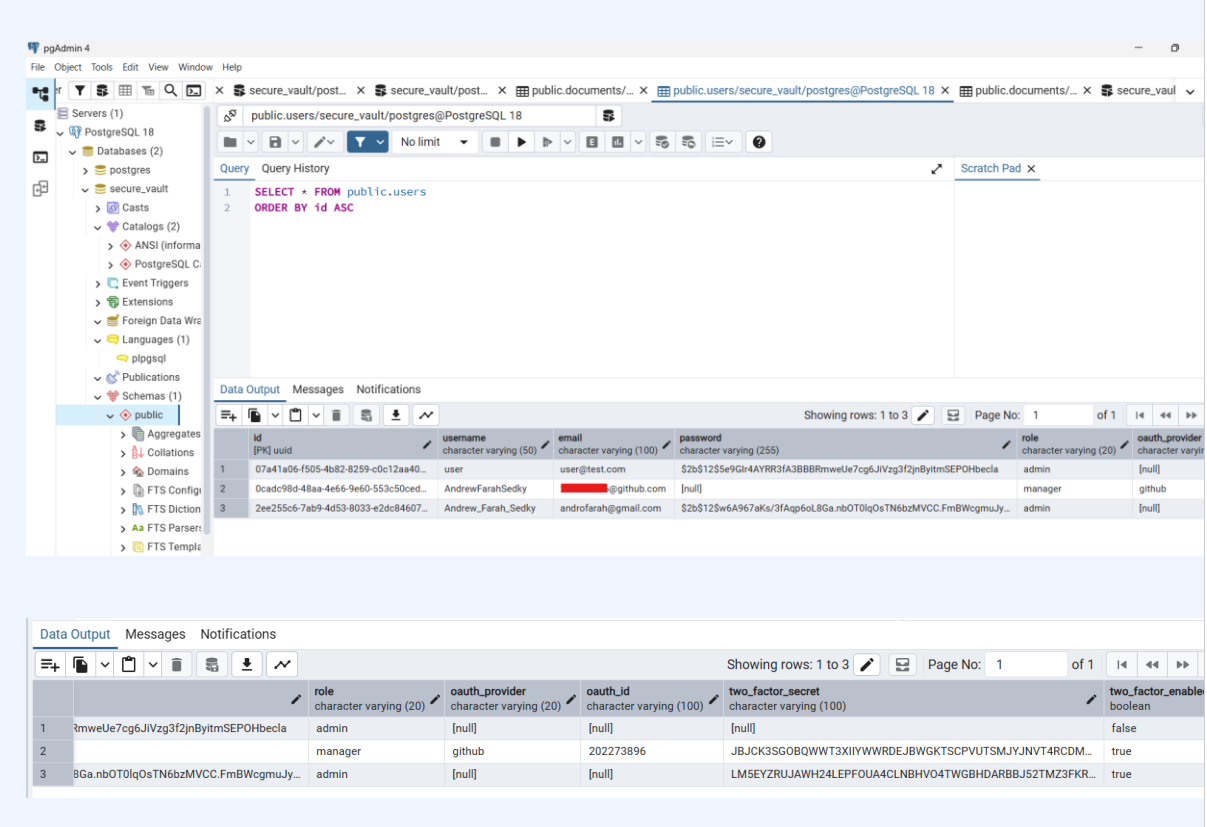
The system uses PostgreSQL with two main tables:

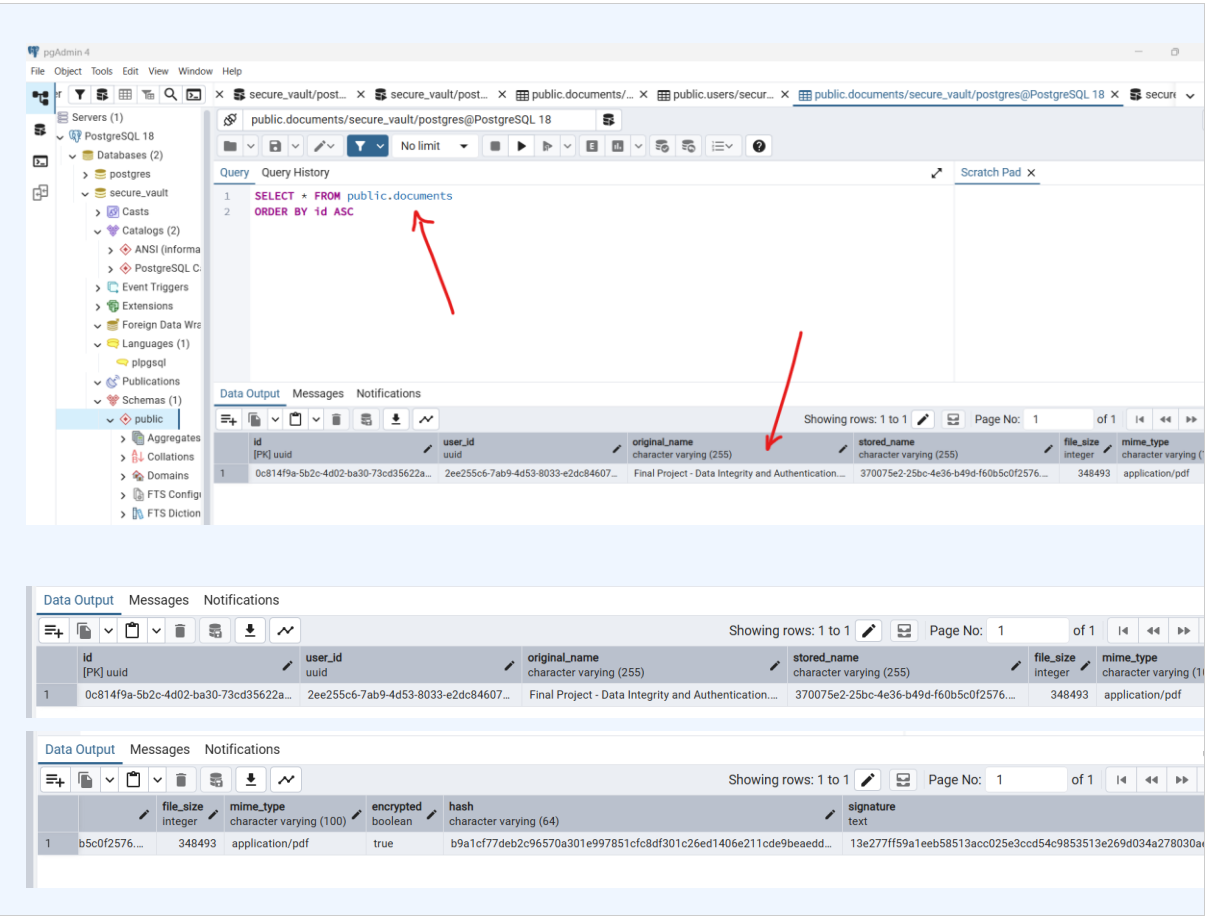
10.1 Users Table

- id — UUID primary key
- username, email — unique identifiers
- password — bcrypt hashed (nullable for OAuth users)
- role — 'admin', 'manager', or 'user'
- oauth_provider, oauth_id — for GitHub/Google login
- two_factor_secret, two_factor_enabled — for 2FA

10.2 Documents Table

- id — UUID primary key
- user_id — foreign key to users
- original_name, stored_name — file names
- file_size, mime_type — metadata
- hash — SHA-256 of the original file
- signature — RSA digital signature of the hash


pgAdmin 4
File Object Tools Edit View Window Help
Servers (1)
PostgreSQL 18
Databases (2)
postgres
secure_vault
public.users/secure_vault/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
SELECT * FROM public.users
ORDER BY id ASC
Data Output Messages Notifications
Showing rows: 1 to 3
Page No: 1 of 1
id [PK] uuid
username character varying (50)
email character varying (100)
password character varying (255)
role character varying (20)
oauth_provider
1 07a41a06-f505-4b82-8259-c0c12aa40... user user@test.com $2b$12$5e9Glr4AYRR3fA3BBBRmweUe7cg6JiVzg3f2jnByitmSEPOHbecla admin [null]
2 0cadc98d-48aa-4e66-9e60-553c50ced... AndrewFarahSedky [redacted]@github.com [null] manager github
3 2ee255c6-7ab9-4d53-8033-e2dc84607... Andrew_Farah_Sedky androfarah@gmail.com $2b$12$w6A967aKs/3fAqp6oL8Ga.nbOT0lqOsTN6bzMVCC.FmBWcgmuJy... admin [null]
Data Output Messages Notifications
Showing rows: 1 to 3
Page No: 1 of 1
role character varying (20)
oauth_provider character varying (20)
oauth_id character varying (100)
two_factor_secret character varying (100)
two_factor_enable boolean
1 RmweUe7cg6JiVzg3f2jnByitmSEPOHbecla admin [null] [null] [null] false
2 manager github 202273896 JBJCK3SGOBQWWT3XIIYWWRDEJBWGKTSCPVUTSMJYJNVT4RCDM... true
3 8Ga.nbOT0lqOsTN6bzMVCC.FmBWcgmuJy... admin [null] [null] LM5EYZRUJAWH24LEPFOUA4CLNBHVO4TWGBHDARBBJ52TMZ3FKR... true


pgAdmin 4
File Object Tools Edit View Window Help
Servers (1)
PostgreSQL 18
Databases (2)
postgres
secure_vault
public.documents/secure_vault/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
SELECT * FROM public.documents
ORDER BY id ASC
Data Output Messages Notifications
Showing rows: 1 to 1
Page No: 1 of 1
id [PK] uuid
user_id uuid
original_name character varying (255)
stored_name character varying (255)
file_size integer
mime_type character varying
1 0c814f9a-5b2c-4d02-ba30-73cd35622a... 2ee255c6-7ab9-4d53-8033-e2dc84607... Final Project - Data Integrity and Authentication.... 370075e2-25bc-4e36-b49d-f60b5c0f2576.... 348493 application/pdf
Data Output Messages Notifications
Showing rows: 1 to 1
Page No: 1 of 1
id [PK] uuid
user_id uuid
original_name character varying (255)
stored_name character varying (255)
file_size integer
mime_type character varying (10
1 0c814f9a-5b2c-4d02-ba30-73cd35622a... 2ee255c6-7ab9-4d53-8033-e2dc84607... Final Project - Data Integrity and Authentication.... 370075e2-25bc-4e36-b49d-f60b5c0f2576.... 348493 application/pdf
Data Output Messages Notifications
Showing rows: 1 to 1
Page No: 1 of 1
file_size integer
mime_type character varying (100)
encrypted boolean
hash character varying (64)
signature text
1 b5c0f2576.... 348493 application/pdf true b9a1cf77deb2c96570a301e997851cfc8df301c26ed1406e211cde9beaedd... 13e277ff59a1eeb58513acc025e3ccd54c9853513e269d034a278030a

11. How to Run the Project

Prerequisites

- Node.js v20+
- PostgreSQL v15+
- Git

Steps

1. Clone the repository from GitHub
2. Create the database: run database/schema.sql in pgAdmin
3. Configure backend/.env with your database credentials and OAuth keys
4. Install backend dependencies: cd backend && npm install
5. Run backend: npm run dev (starts on port 5000)
6. Install frontend dependencies: cd frontend && npm install
7. Run frontend: npm run dev (opens on https://localhost:5173)

12. Conclusion

The Secure Document Vault System successfully demonstrates a comprehensive implementation of modern security concepts in a real-world web application. The project integrates multiple security layers working together to protect sensitive documents and user data.

Key security achievements include end-to-end encrypted document storage, multi-factor authentication, granular role-based access control, cryptographic integrity verification, and secure HTTPS communication. The Wireshark demonstration clearly illustrates the critical importance of transport layer security.

This project simulates the security architecture found in real enterprise document management systems and provides a solid foundation for understanding applied cryptography and authentication in modern web development.

Github Repository Link :

https://github.com/AndrewFarahSedky/Secure-Document-Vault-System.git